National Laboratory of the Rockies

Codebase Structure and Workflow for the Anderson Impurity Model Project

Azzan Porter

National Laboratories of Rockies (AIM Quantum Computing)

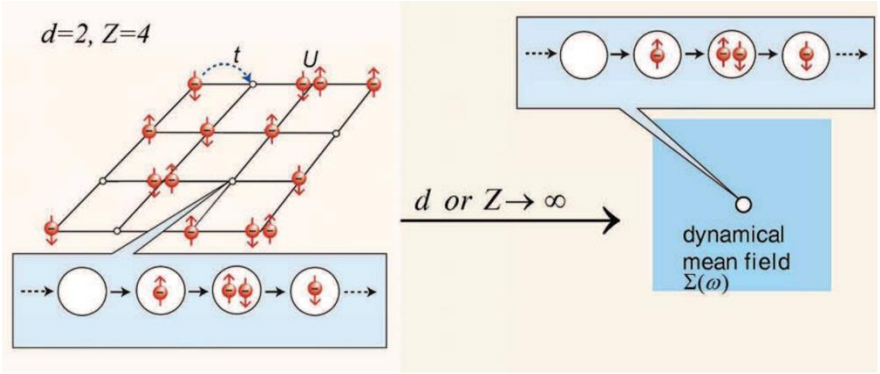
- Contents

1. Anderson Impurity Model
2. Mathematical Formulation
3. Codebase Flowchart Diagram
4. dmft.py Code Breakdown
5. initialize_system() Code Breakdown
6. anderson_impurity_model.py Code Breakdown
7. n_site_graph_creation.py Code Breakdown
8. exact.py gs
9. exact.py gf
10. vqe.py gs
11. vqe.py gf
12. Comparing Exact and VQE Results
13. Next Steps
14. Sources
15. Questions

What is the Anderson Impurity Model?

A small interacting site known as the impurity is related to a larger collection of surrounding noninteracting sites known as the bath in the Anderson Impurity Model, which is frequently abbreviated as AIM.

In this project, the bath is the surrounding environment that the impurity can exchange particles with, and the impurity is the significant quantum system we are interested in.



Source: *Dynamical Mean-Field Theory of Strongly Correlated Electron Systems* by Dieter Vollhardt

$$H_{tot} = H_{imp} + H_{hyb} + H_{bath}$$

$$G_{local}(i\omega_n) = \sum_k \frac{1}{i\omega_n - \epsilon_k + \mu - \sum i\omega_n}$$

Mathematical Formulation

Hamiltonian:

$$H_{AIM} = H_{imp} + H_{bath} + H_{hyb}$$

$$H_{\mathrm{AIM}} = \sum_{\sigma} \epsilon_d d_\sigma^\dagger d_\sigma + U n_{d\uparrow} n_{d\downarrow} + \sum_{k\sigma} \epsilon_k c_{k\sigma}^\dagger c_{k\sigma} + \sum_{k\sigma} V_k \left(c_{k\sigma}^\dagger d_\sigma + d_\sigma^\dagger c_{k\sigma} \right)$$

Ground State & VQE:

$$E_0 = \min_{\psi} < \psi | H | \psi >$$

Green's Function:

$$G_{ij}^R(\omega) = \left\langle \psi_0 | c_i \frac{1}{\omega + E_0 - H + i\eta} c_j^\dagger | \psi_0 \right\rangle + \left\langle \psi_0 | c_j^\dagger \frac{1}{\omega - E_0 + H + i\eta} c_i | \psi_0 \right\rangle$$

$G(\omega) = G^+(\omega) - G^-(\omega)$ (Simplified)

Hamiltonian: Represents the total energy of the AIM system.

Ground State: The lowest-energy state of the Hamiltonian.

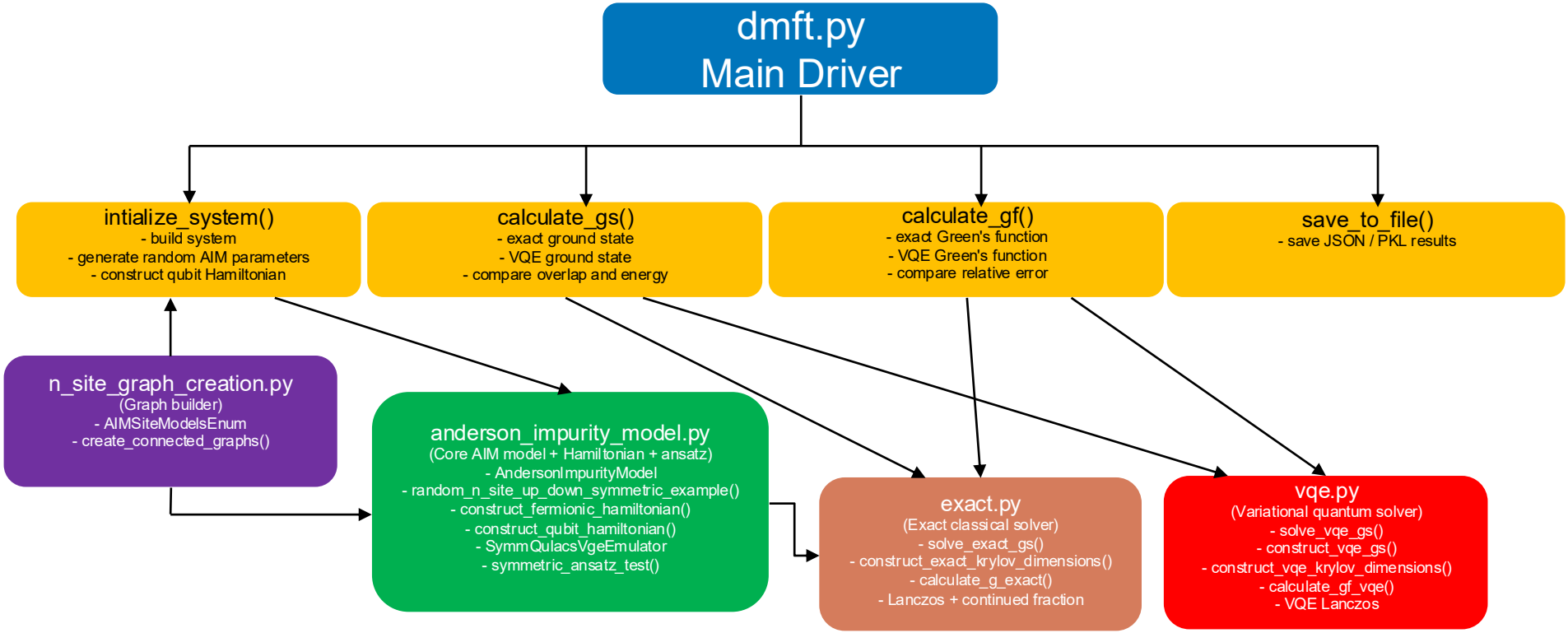
VQE: Approximates the ground state by minimizing energy.

Green's Function: Describes the system's response when an electron is added or removed.

Main Packages and Libraries Used

Package/Library	Function
Numpy	Handles numerical arrays, vectors, matrix-style operations, and energy/error calculations.
SciPy	Provides scientific-computing tools, especially sparse matrices, eigenvalue solvers, and linear algebra.
Matplotlib	Used to plot or visualize results.
NetworkX	Builds graph structures for impurity and bath sites.
OpenFermion	Represents fermionic operators and helps convert the AIM Hamiltonian into qubit form.
Qulacs	Simulates quantum circuits used in VQE.
Qulacsvis	Visualizes Qulacs quantum circuits.
Noisyopt	Optimizes VQE parameters when the objective function may be noisy.

Codebase Flowchart Diagram



Overall pipeline: build AIM → exact ground state vs VQE ground state → exact Green's function vs VQE Green's function → compare errors → save results

dmft.py Code Breakdown

The imports show that dmft.py connects the major parts of the project. It uses anderson_impurity_model.py to build the model, exact.py for exact calculations, vqe.py for variational calculations, and n_site_graph_creation.py for the graph structure.

- main(): reads command-line inputs
- initialize_system(): builds the model and qubit Hamiltonian
- calculate_gs(): compare exact and VQE ground states
- calculate_gf(): compares exact and VQE Green's functions
- save_to_file(): stores the final results

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

initialization_system() Code Breakdown

- This part of the code turns the user's chosen system size and seed into an actual Anderson Impurity Model.
- First, it generates the model parameters. Then it creates the model object, identifies the spin-up and spin-down qubit indices, and finally constructs the qubit Hamiltonian that will be used by the exact and VQE solvers.

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
Random Seed	Makes the generated AIM parameters reproducible.
AIM parameter generator	Creates impurity energy, bath energies, hopping terms, and interaction strength.
AndersonImpurityModel	Stores the physical model.
Spin-up / spin-down indices	Identifies how spin sectors are represented in the qubit system.
Qubit Hamiltonian constructor	Converts the AIM Hamiltonian into the form needed by exact and VQE calculations.

anderson_impurity_model.py Code Breakdown

- This file takes the physical parameters of the Anderson Impurity Model and turns them into a Hamiltonian.
- First, the model is stored in an AndersonImpurityModel object. Then the code constructs the fermionic Hamiltonian, which represents the original electron system.
- After that, it converts the fermionic Hamiltonian into a qubit Hamiltonian so it can be used by quantum simulation methods like VQE.

Component	Role
OpenFermion	Represents creation and annihilation operators for electrons.
Fermionic Hamiltonian tools	Build the original electron-based Hamiltonian.
Qubit mapping tools	Convert the fermionic Hamiltonian into a qubit Hamiltonian.
Qulacs	Supports quantum-circuit simulation for VQE.
Ansatz construction	Builds the parameterized circuit used to prepare trial states.
AIM model class	Stores impurity, bath, interaction, and hopping parameters.

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

n_site_graph_creation.py Code Breakdown

- This code creates the graph layout for the Anderson Impurity Model.
- The enum chooses a predefined model size, such as one impurity with several bath sites.
- Then create_connected_graphs() builds the graph objects used later by the Hamiltonian construction and the VQE ansatz.

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
NetworkX	Creates and stores graph structures.
AIMSiteModelsEnum	Selects predefined impurity-bath layouts.
Graph nodes	Represent impurity and bath sites.
Graph edges	Represent connections/hybridization between sites.
Connected graph builder	Ensures the AIM system is represented as one connected structure.

exact.py gs

The Hamiltonian represents the total energy of the AIM. Exact diagonalization solves the eigenvalue equation.

$$H|\psi_n\rangle = E|\psi_n\rangle$$

After find the solutions it orders them so it can find the ground state energy E_0 and the corresponding eigenvector is the ground-state wavefunction $|\psi_0\rangle$.

Matrix Diagonalization: $H = \begin{pmatrix} H_{00} & \cdots & H_{0m} \\ \vdots & \ddots & \vdots \\ H_{n0} & \cdots & H_{nm} \end{pmatrix}$

Diagonalization: $E_0, E_1, E_2, \ldots$ where E_0 is the ground state.

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
Numpy	Handles vectors, arrays, and numerical values.
SciPy sparse	Stores large Hamiltonian matrices efficiently.
Eigenvalue solver	Finds the allowed energy values of the Hamiltonian and the quantum states that go with them.
Ground-state selection	Chooses the lowest eigenvalue, E_0, as the ground-state energy.
Eigenvector output	Gives the exact ground-state wavefunction

exact.py gf

Green's Functions: $G(\omega) = G^{+}(\omega) - G^{-}(\omega)$

Lanczos Theory: $\{|\phi\rangle,\ H|\phi\rangle,\ H^2|\phi\rangle, \dots\}$

Lanczos coefficients: $G(\omega) = \cfrac{1}{\omega - a_0 - \cfrac{b_1^2}{\omega - a_1 - \cfrac{b_2^2}{\omega - a_2 - \cdots}}}$

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
Numpy	Handles vectors, arrays, and numerical values.
SciPy sparse	Stores large Hamiltonian matrices efficiently.
Lanczos method	Builds a Krylov basis from repeated action of H.
Lanczos coefficients	Produces the a_i and b_i values used in the continued fraction.
Continued fraction	Computes the Green's function from the Lanczos coefficients.

vqe.py gs

- The exact solver tries to solve: $H|\psi_0\rangle = E|\psi_0\rangle$
- The goal is to minimize the energy:

$$E_0 \approx \min_{\theta} \langle\psi(\theta)|H|\psi(\theta)\rangle$$

So vqe.py is not only minimizing energy. It is also tracking the particle number and spin sector of the VQE result.

- Exact diagonalization finds the true lowest eigenvalue directly.
- VQE instead searches only through the states that can be produced by the chosen ansatz circuit: $|\psi(\theta)\rangle$

Component	Role
Qulacs	Simulates the parameterized quantum circuit.
Ansatz circuit	Prepares trial states
Optimizer / Noisyopt	Updates parameters θ to reduce the energy.
Hamiltonian expectation value	Computes $\|\psi(\theta)\rangle$
Particle/spin checks	Verifies whether the VQE state is in the correct physical sector.

vqe.py gf

Green's Functions: $G(\omega) = G^{+}(\omega) - G^{-}(\omega)$

Lanczos Theory: $\{|\phi\rangle,\ H|\phi\rangle,\ H^2|\phi\rangle, \ldots\}$

Lanczos coefficients: $G(\omega) = \cfrac{1}{\omega - a_0 - \cfrac{b_1^2}{\omega - a_1 - \cfrac{b_2^2}{\omega - a_2 - \cdots}}}$

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
Qulacs	Prepares VQE-based quantum states.
VQE ground state	Provides the approximate starting state for Green's function calculations.
Krylov/Lanczos tools	Build an approximate Krylov basis using the VQE state.
Continued fraction	Computes the approximate Green's function.
Error comparison tools	Compare VQE Green's function results against exact results.

Comparing Exact and VQE Results

- The project compares the exact ground state and the VQE ground state using two main quantities: E_{exact}, E_{VQE}

$$(GS)\ Relative\ Error: \frac{|E_{exact} - E_{VQE}|}{|E_{exact}|}$$

- Wavefunction Overlap:

$$Overlap = |\langle \psi_{VQE} | \psi_{exact} \rangle|$$

- After the Green's functions are computed, the project compares: G_{exact}, G_{VQE}

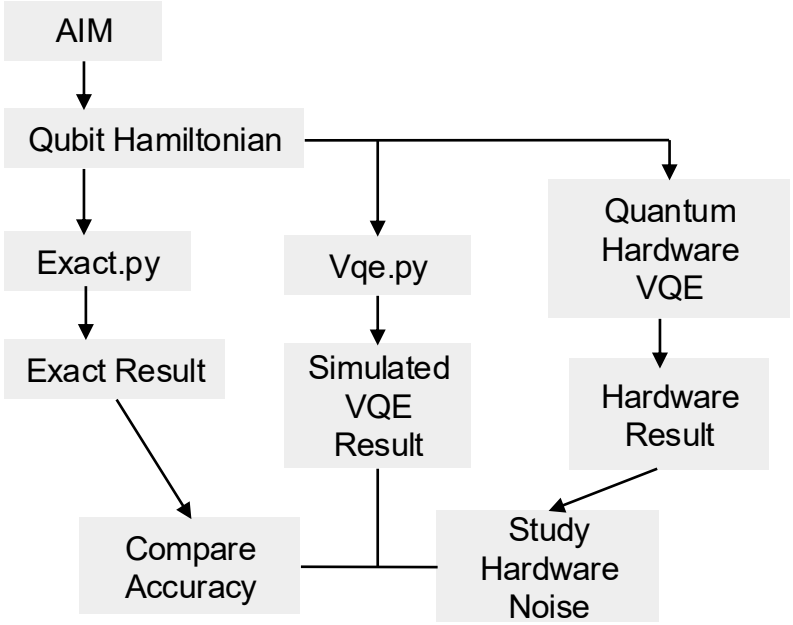
$$(GF)\ Relative\ Error: \frac{||G_{VQE} - G_{exact}||}{||G_{exact}||}$$

Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

Component	Role
E_{exact}	Ground-state energy from exact diagonalization.
E_{VQE}	Approximate ground-state energy from VQE.
GS relative error	How close the VQE energy is to the exact energy.
Wavefunction overlap	How similar the VQE state is to the exact ground state.
G_{exact}	Green's function from exact calculation.
G_{VQE}	Green's function from VQE-based calculation.
GF relative error	How close the VQE Green's function is to the exact Green's function.

Next Steps

- The current project uses classical simulation to compare: Exact classical method vs Simulated VQE method.
- A natural next step would be to run the VQE part on an actual quantum computer and compare: Exact classical result vs Quantum hardware VQE result.
- The optimizer still updates θ, but the energy measurements would come from the quantum device.
- How close does the quantum hardware result get to the exact classical benchmark?



Source: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py

New Idea using Quantum Phase Estimation

Method	Exact Solver	QPE
Type	Classical Algorithm	Quantum Algorithm
Main goal	Finding the eigenvalues by diagonalizes the Hamiltonian matrix	Estimating the energies by measuring the phase from quantum time evolution
Main equation	$H\|\psi\rangle = E\|\psi\rangle$	$U\|\psi\rangle = e^{-iEt}\|\psi\rangle$
Hamiltonian use	Uses H directly as a matrix	Uses H inside the time-evolution operator $U = e^{-iHt}$
Main limitation	Becomes expensive as the Hamiltonian matrix gets large	Requires deeper circuits and more reliable quantum hardware

In this project, the exact solver stays classical and provides the benchmark. QPE is the closest quantum analogue because it estimates Hamiltonian eigenvalues on a quantum computer, but it requires deeper circuits than VQE.

Sources & Contributors

- **Vollhardt, D. (2019). *Dynamical Mean-Field Theory of Strongly Correlated Electron Systems*. arXiv:1910.12650v2.**
- GitHub: https://github.com/NatLabRockies/qcaob-aim/blob/main/dmft.py
- Contributors: Cody James Winkleblack, Derek Vigil-Fowler, Taylor Aubry, Jacob Clary



National Laboratory of the Rockies

Photo from Getty Images 181828180